

TP 6: Apache Yarn

Université / École :

- Université Ibn Tofail, Kénitra

Encadrement pédagogique :

- Pr. Hajar Filali

Rédigé par :

- Mohammed Jebrou
- Hajar Farid

Année universitaire :

- 2025–2026

Introduction

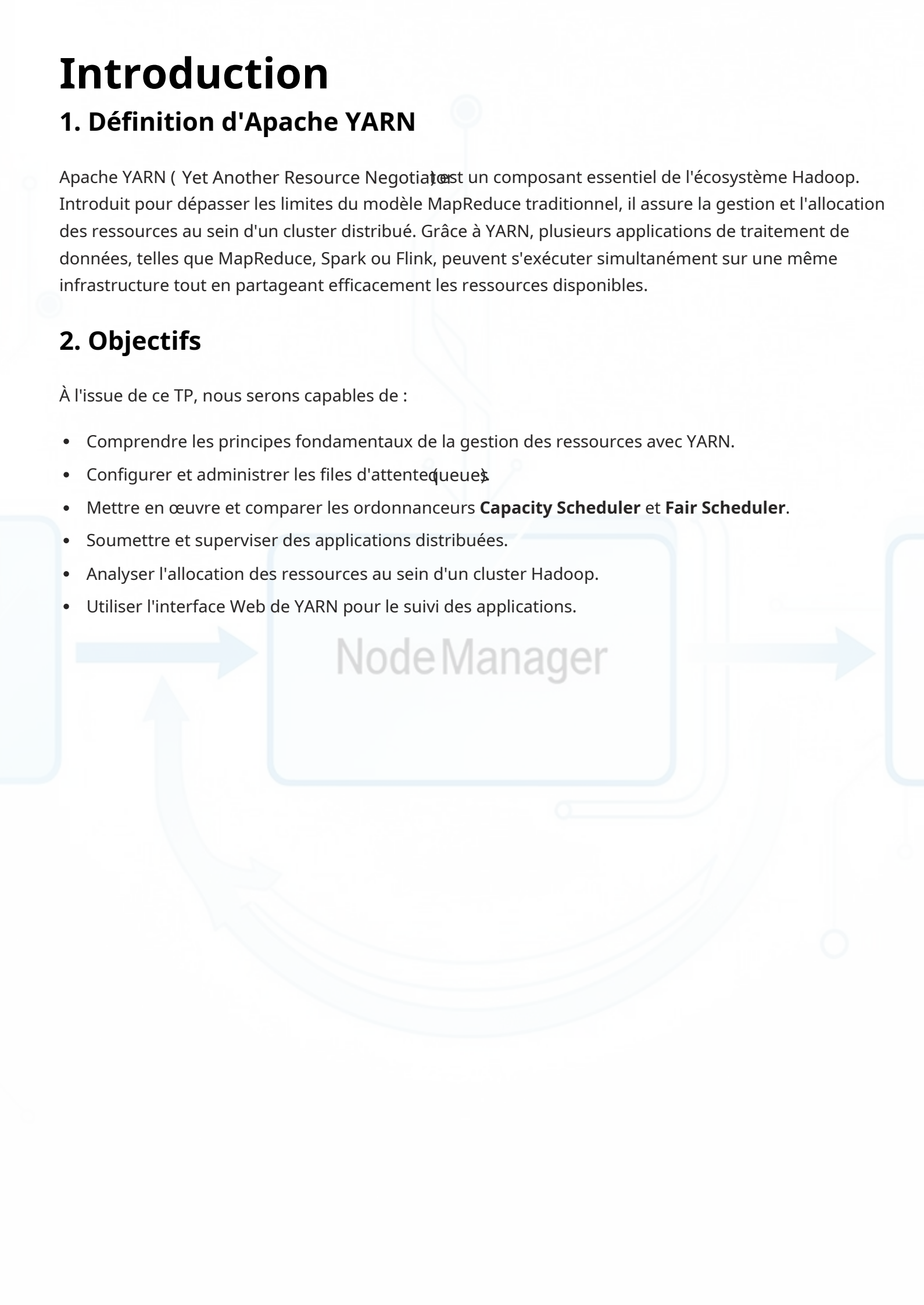
1. Définition d'Apache YARN

Apache YARN (Yet Another Resource Negotiator) est un composant essentiel de l'écosystème Hadoop. Introduit pour dépasser les limites du modèle MapReduce traditionnel, il assure la gestion et l'allocation des ressources au sein d'un cluster distribué. Grâce à YARN, plusieurs applications de traitement de données, telles que MapReduce, Spark ou Flink, peuvent s'exécuter simultanément sur une même infrastructure tout en partageant efficacement les ressources disponibles.

2. Objectifs

À l'issue de ce TP, nous serons capables de :

- Comprendre les principes fondamentaux de la gestion des ressources avec YARN.
- Configurer et administrer les files d'attente (queues).
- Mettre en œuvre et comparer les ordonnanceurs **Capacity Scheduler** et **Fair Scheduler**.
- Soumettre et superviser des applications distribuées.
- Analyser l'allocation des ressources au sein d'un cluster Hadoop.
- Utiliser l'interface Web de YARN pour le suivi des applications.



The diagram illustrates the Node Manager component within a Hadoop cluster. A central box labeled 'Node Manager' is shown with a light blue border. To its left, a blue arrow points towards it, and to its right, another blue arrow points away from it. Below the 'Node Manager' box, there are several curved, light blue arrows that suggest a flow or interaction between different parts of the system, possibly representing the distribution of resources or the execution of tasks across multiple nodes.

Node Manager

Prérequis et vérification de l'environnement

1. Vérification de Java et Hadoop

Avant le démarrage des services Hadoop, il est nécessaire de vérifier la disponibilité de Java ainsi que la bonne configuration de Hadoop et des variables d'environnement associées.

```
mohammed@ROG-Strix:~/HADOOP_REPO/yarn
> java -version
openjdk version "1.8.0_482"
OpenJDK Runtime Environment (build 1.8.0_482-8u482-ga~us1-0ubuntu1~24.04-b08)
OpenJDK 64-Bit Server VM (build 25.482-b08, mixed mode)
> hadoop version
Hadoop 2.7.1
Subversion https://git-wip-us.apache.org/repos/asf/hadoop.git -r 15ecc87ccf4a0228f35af08fc56de536e6ce657a
Compiled by jenkins on 2015-06-29T06:04Z
Compiled with protoc 2.5.0
From source with checksum fc0a1a23fc1868e4d5ee7fa2b28a58a
This command was run using /home/mohammed/hadoop/share/hadoop/common/hadoop-common-2.7.1.jar
> echo $HADOOP_COMMON_HOME
/home/mohammed/hadoop
> echo $JAVA_HOME
/usr/lib/jvm/java-8-openjdk-amd64
yarn >
```

2. Démarrer HDFS et YARN

```
mohammed@ROG-Strix:~/HADOOP_REPO/yarn
> start-dfs.sh
Starting namenodes on [localhost]
localhost: starting namenode, logging to /home/mohammed/hadoop/logs/hadoop-mohammed-namenode-ROG-Strix.out
localhost: starting datanode, logging to /home/mohammed/hadoop/logs/hadoop-mohammed-datanode-ROG-Strix.out
Starting secondary namenodes [0.0.0.0]
0.0.0.0: starting secondarynamenode, logging to /home/mohammed/hadoop/logs/hadoop-mohammed-secondarynamenode-ROG-Strix.out
> start-yarn.sh
starting yarn daemons
starting resourcemanager, logging to /home/mohammed/hadoop/logs/yarn-mohammed-resourcemanager-ROG-Strix.out
localhost: starting nodemanager, logging to /home/mohammed/hadoop/logs/yarn-mohammed-nodemanager-ROG-Strix.out
> jps
12883 NodeManager
12565 ResourceManager
11973 NameNode
13030 Jps
10918
12363 SecondaryNameNode
12140 DataNode
yarn >
```

3. Préparation des données du test sur HDFS

Afin de tester le fonctionnement de YARN et du système de fichiers distribué HDFS, un fichier texte de test est créé localement puis transféré vers HDFS. Les commandes `mkdir`, `put`, `ls` et `cat` permettent respectivement de créer un répertoire, charger le fichier, vérifier sa présence et consulter son contenu.

```
mohammed@ROG-Strix:~/HADOOP_REPO/yarn
> hdfs dfs -mkdir -p /yarn-demo/input
> echo "hadoop yarn scheduler capacity fair resource management cluster big data" > test.txt
> hdfs dfs -put test.txt /yarn-demo/input/
> hdfs dfs -ls /yarn-demo/input/
Found 1 items
-rw-r--r--  1 mohammed supergroup      74 2026-04-22 11:55 /yarn-demo/input/test.txt
> hdfs dfs -cat /yarn-demo/input/test.txt
hadoop yarn scheduler capacity fair resource management cluster big data
yarn )
```

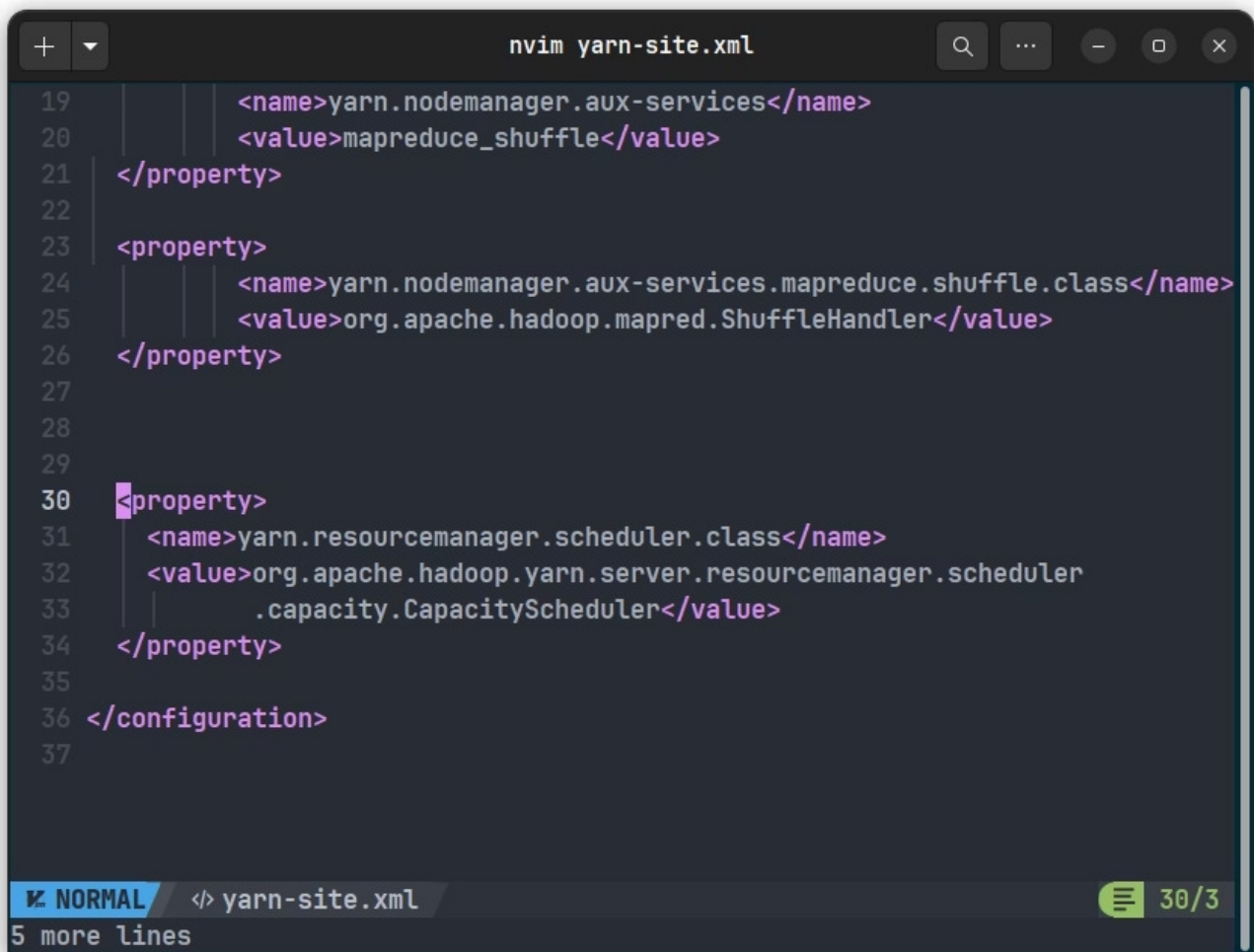
4. Configuration du fichier capacity-scheduler.xml

Cette étape consiste à configurer le fichier `capacity-scheduler.xml` afin de définir les différentes files d'attente du Capacity Scheduler. Chaque file se voit attribuer une part des ressources du cluster, ce qui permet d'assurer une répartition équilibrée des ressources entre les applications exécutées sur YARN.

```
nvim etc/hadoop/capacity-scheduler.xml
mohammed@ROG-Strix:~/Desktop/COUR... x nvim etc/hadoop/capacity-scheduler... x
22 <!-- engineering queue -->
23 <property>
24   <name>yarn.scheduler.capacity.root.engineering.capacity</name>
25   <value>50</value>
26 </property>
27 <property>
28   <name>yarn.scheduler.capacity.root.engineering.maximum-capacity</name>
29   <value>70</value>
30 </property>
31 <property>
32   <name>yarn.scheduler.capacity.root.engineering.state</name>
33   <value>RUNNING</value>
34 </property>
35
36 <!-- research queue -->
37 <property>
38   <name>yarn.scheduler.capacity.root.research.capacity</name>
39   <value>30</value>
40 </property>
41 <property>
42   <name>yarn.scheduler.capacity.root.research.maximum-capacity</name>
43   <value>50</value>
44 </property>
45 </configuration>
```

5. Activation du Capacity Scheduler

Afin d'utiliser le Capacity Scheduler comme mécanisme d'allocation des ressources dans YARN, il est nécessaire de modifier le fichier `yarn-site.xml` en spécifiant la classe du planificateur à utiliser.



```
nvim yarn-site.xml
19      <name>yarn.nodemanager.aux-services</name>
20      <value>mapreduce_shuffle</value>
21    </property>
22
23    <property>
24      <name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
25      <value>org.apache.hadoop.mapred.ShuffleHandler</value>
26    </property>
27
28
29
30    <property>
31      <name>yarn.resourcemanager.scheduler.class</name>
32      <value>org.apache.hadoop.yarn.server.resourcemanager.scheduler
33        .capacity.CapacityScheduler</value>
34    </property>
35
36  </configuration>
37
```

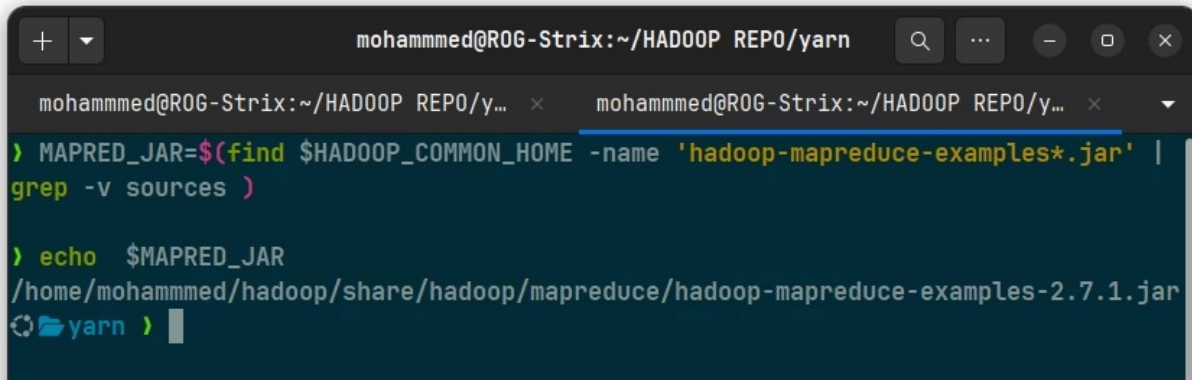
5 more lines

6 Redémarrage des services et exécution des jobs MapReduce

Après la modification de la configuration, les services YARN sont redémarrés afin d'appliquer les changements. Un exemple MapReduce est ensuite utilisé pour vérifier le bon fonctionnement du Capacity Scheduler.

6.1 Identification du package d'exemples MapReduce

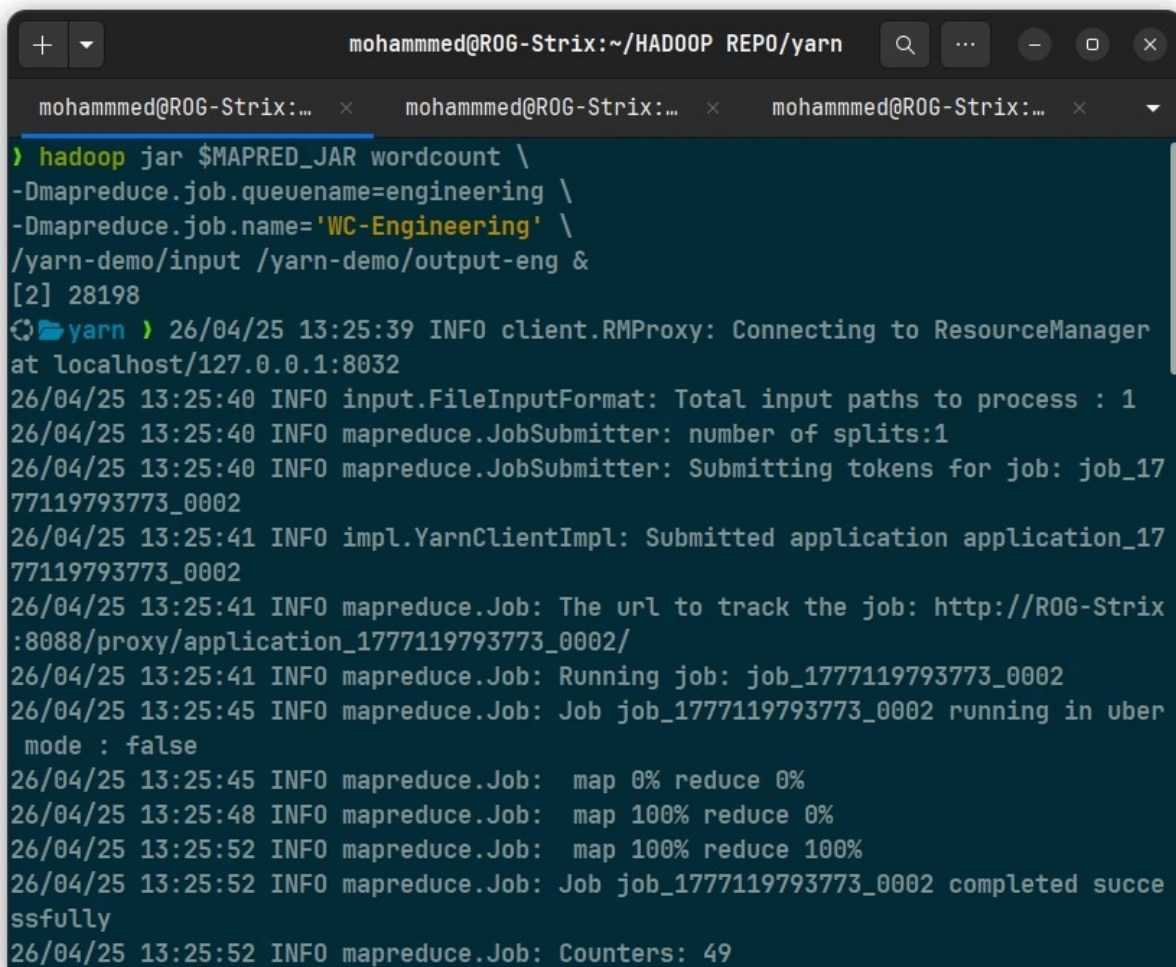
La commande suivante permet de localiser le fichier JAR contenant les exemples MapReduce fournis avec Hadoop. Ce package sera utilisé pour exécuter des tests sur l'infrastructure YARN.



```
mohammed@ROG-Strix:~/HADOOP_REPO/yarn
> MAPRED_JAR=$(find $HADOOP_COMMON_HOME -name 'hadoop-mapreduce-examples*.jar' |
grep -v sources )
> echo $MAPRED_JAR
/home/mohammed/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-2.7.1.jar
yarn >
```

6.2 Soumission d'un job dans la file « Engineering »

Une première application WordCount est exécutée dans la file d'attente **Engineering** afin de vérifier son bon fonctionnement et l'allocation des ressources qui lui sont attribuées.



```
mohammed@ROG-Strix:~/HADOOP_REPO/yarn
> hadoop jar $MAPRED_JAR wordcount \
-Dmapreduce.job.quevename=engineering \
-Dmapreduce.job.name='WC-Engineering' \
/yarn-demo/input /yarn-demo/output-eng &
[2] 28198
yarn > 26/04/25 13:25:39 INFO client.RMProxy: Connecting to ResourceManager
at localhost/127.0.0.1:8032
26/04/25 13:25:40 INFO input.FileInputFormat: Total input paths to process : 1
26/04/25 13:25:40 INFO mapreduce.JobSubmitter: number of splits:1
26/04/25 13:25:40 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_17
77119793773_0002
26/04/25 13:25:41 INFO impl.YarnClientImpl: Submitted application application_17
77119793773_0002
26/04/25 13:25:41 INFO mapreduce.Job: The url to track the job: http://ROG-Strix
:8088/proxy/application_1777119793773_0002/
26/04/25 13:25:41 INFO mapreduce.Job: Running job: job_1777119793773_0002
26/04/25 13:25:45 INFO mapreduce.Job: Job job_1777119793773_0002 running in uber
mode : false
26/04/25 13:25:45 INFO mapreduce.Job: map 0% reduce 0%
26/04/25 13:25:48 INFO mapreduce.Job: map 100% reduce 0%
26/04/25 13:25:52 INFO mapreduce.Job: map 100% reduce 100%
26/04/25 13:25:52 INFO mapreduce.Job: Job job_1777119793773_0002 completed succe
ssfully
26/04/25 13:25:52 INFO mapreduce.Job: Counters: 49
```

6.2 Soumission d'un job dans la file « Research »

Une seconde application WordCount est ensuite soumise à la file d'attente **Research** afin de comparer l'exécution des applications dans différentes files configurées par le Capacity Scheduler.

```
> hadoop jar $MAPRED_JAR wordcount \
-Dmapreduce.job.queueName=research \
-Dmapreduce.job.name='WC-Research' \
/yarn-demo/input /yarn-demo/output-res &
[2] 28726
yarn > 26/04/25 13:27:04 INFO client.RMProxy: Connecting to ResourceManager
at localhost/127.0.0.1:8032
26/04/25 13:27:04 INFO input.FileInputFormat: Total input paths to process : 1
26/04/25 13:27:04 INFO mapreduce.JobSubmitter: number of splits:1
26/04/25 13:27:04 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_17
77119793773_0003
26/04/25 13:27:04 INFO impl.YarnClientImpl: Submitted application application_17
77119793773_0003
26/04/25 13:27:04 INFO mapreduce.Job: The url to track the job: http://ROG-Strix
:8088/proxy/application_1777119793773_0003/
26/04/25 13:27:04 INFO mapreduce.Job: Running job: job_1777119793773_0003
26/04/25 13:27:08 INFO mapreduce.Job: Job job_1777119793773_0003 running in uber
mode : false
26/04/25 13:27:08 INFO mapreduce.Job: map 0% reduce 0%
26/04/25 13:27:13 INFO mapreduce.Job: map 100% reduce 0%
26/04/25 13:27:17 INFO mapreduce.Job: map 100% reduce 100%
26/04/25 13:27:17 INFO mapreduce.Job: Job job_1777119793773_0003 completed succe
ssfully
26/04/25 13:27:17 INFO mapreduce.Job: Counters: 49
```

6.3 Consultation des applications en cours d'exécution

La commande `yarn application -list -appStates ALL` a permis d'afficher l'ensemble des applications gérées par YARN.

```
mohammed@ROG-Strix:~/HADOOP_REPO/yarn
yarn application -list -appStates ALL
26/04/25 13:28:48 INFO client.RMProxy: Connecting to ResourceManager at localhos
t/127.0.0.1:8032
26/04/25 13:28:48 WARN util.NativeCodeLoader: Unable to load native-hadoop libra
ry for your platform... using builtin-java classes where applicable
Total number of applications (application-types: [] and states: [NEW, NEW_SAVING
, SUBMITTED, ACCEPTED, RUNNING, FINISHED, FAILED, KILLED]):2
Application-Id      Application-Name      Application-Type
User                Queue                State                Final-State
Progress            Tracking-URL
application_1777119793773_0003      word count      MAPREDUCE
mohammed            research        FINISHED        SUCCEEDED
100% http://ROG-Strix:19888/jobhistory/job/job_1777119793773_0003
application_1777119793773_0002      word count      MAPREDUCE
mohammed            engineering    FINISHED        SUCCEEDED
100% http://ROG-Strix:19888/jobhistory/job/job_1777119793773_0002
yarn >
```

6.4 Vérification via l'interface Web de YARN

Cluster

About Nodes

Node Labels

Applications

NEW

NEW SAVING

SUBMITTED

ACCEPTED

RUNNING

FINISHED

FAILED

KILLED

Scheduler

Tools

All Applications

Logged in as: drwho

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total	Memory Reserved	VCores Used	VCores Total	VCores Reserved	Active Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes
4	0	0	4	0	0 B	8 GB	0 B	0	8	0	1	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[MEMORY]	<memory:1024, vCores:1>	<memory:8192, vCores:8>

Show 20 entries

ID	User	Name	Application Type	Queue	StartTime	FinishTime	State	FinalStatus	Progress	Tracking UI
application_1777119793773_0005	mohammed	QuasiMonteCarlo	MAPREDUCE	research	Sat Apr 25 13:30:20 +0100 2026	Sat Apr 25 13:30:33 +0100 2026	FINISHED	SUCCEEDED		History
application_1777119793773_0004	mohammed	QuasiMonteCarlo	MAPREDUCE	research	Sat Apr 25 13:29:53 +0100 2026	Sat Apr 25 13:30:07 +0100 2026	FINISHED	SUCCEEDED		History
application_1777119793773_0003	mohammed	word count	MAPREDUCE	research	Sat Apr 25 13:27:04 +0100 2026	Sat Apr 25 13:27:15 +0100 2026	FINISHED	SUCCEEDED		History
application_1777119793773_0002	mohammed	word count	MAPREDUCE	engineering	Sat Apr 25 13:25:40 +0100 2026	Sat Apr 25 13:25:51 +0100 2026	FINISHED	SUCCEEDED		History

Showing 1 to 4 of 4 entries

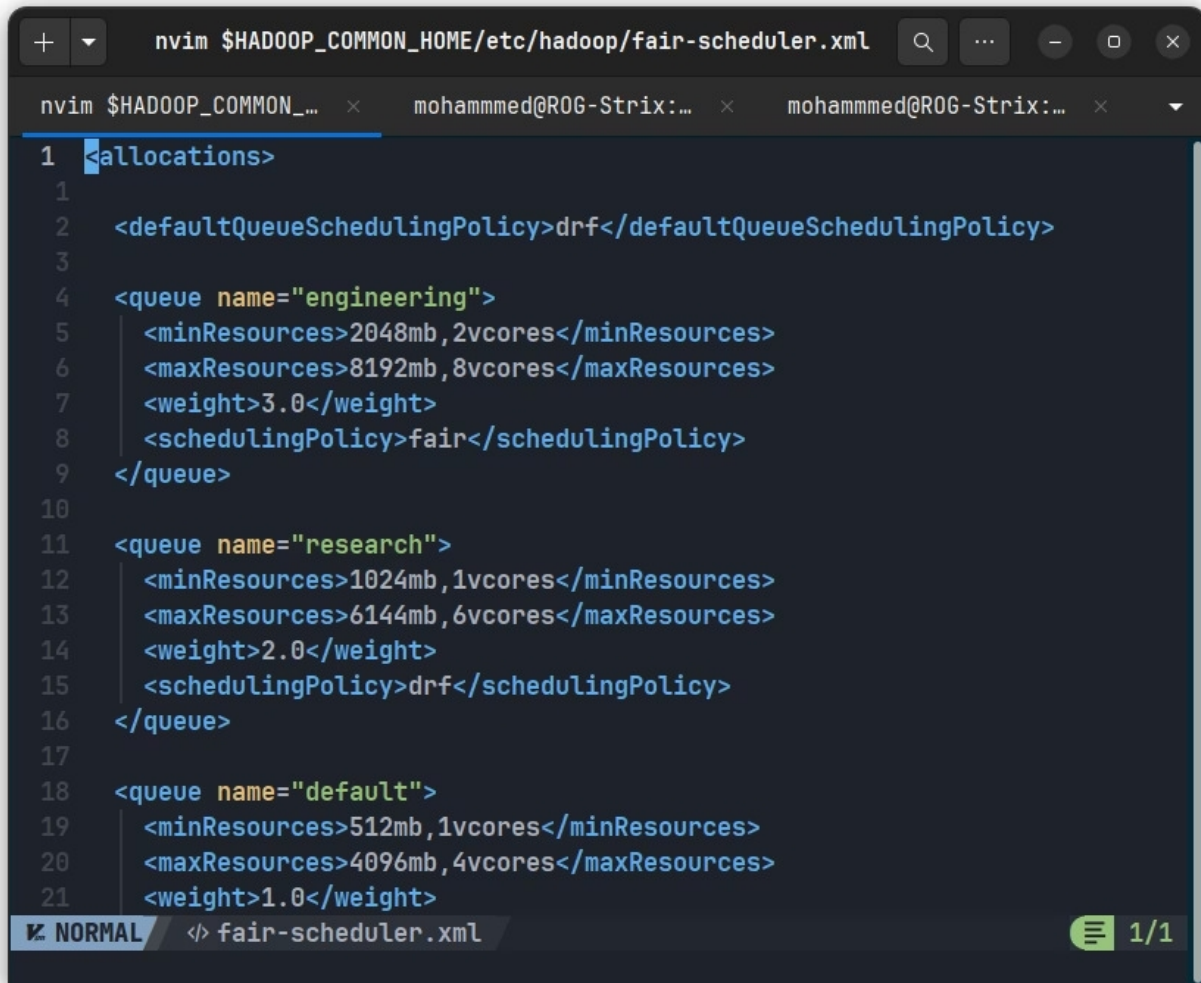
First Previous 1 Next Last

Les résultats observés à travers la ligne de commande et l'interface Web de YARN confirment que les jobs ont été correctement soumis aux files d'attente configurées (**Engineering** et **Research**) et exécutés avec succès. Cette étape valide le bon fonctionnement du **Capacity Scheduler** ainsi que la gestion des ressources par YARN.

Fair Scheduler avec DRF et Prémemption

1. création de fichier fair-scheduler.xml

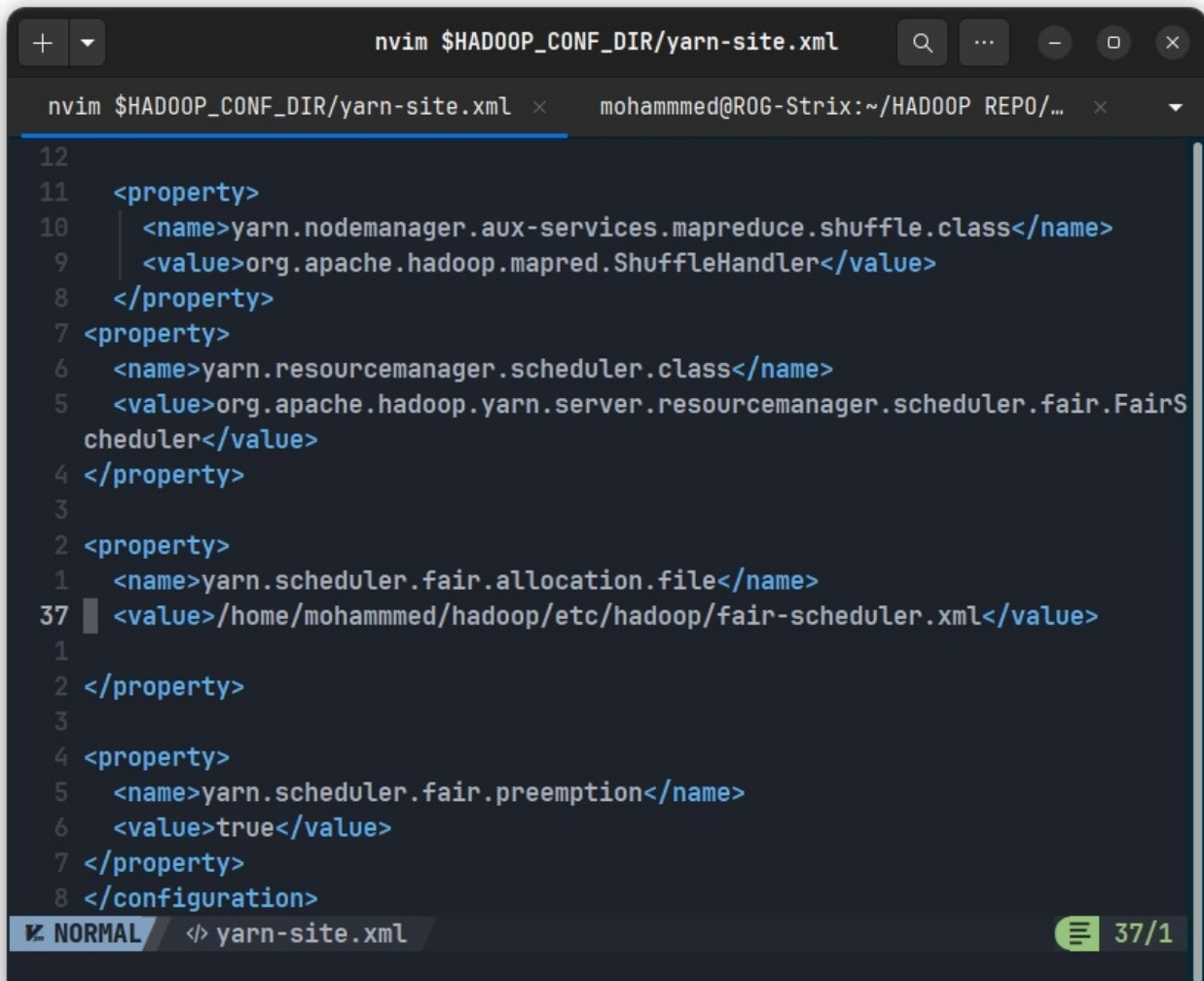
Le fichier `fair-scheduler.xml` est créé afin de configurer le **Fair Scheduler** de YARN. Contrairement au Capacity Scheduler, le Fair Scheduler vise à répartir équitablement les ressources du cluster entre les différentes applications et files d'attente. Dans cette configuration, plusieurs files d'attente sont définies avec des ressources minimales, maximales et des poids (`weight`) spécifiques afin de contrôler l'allocation des ressources.

A screenshot of a code editor window titled 'nvim \$HADOOP_COMMON_HOME/etc/hadoop/fair-scheduler.xml'. The editor shows the XML configuration for the Fair Scheduler. The configuration includes a root element 'allocations' containing three queue definitions: 'engineering', 'research', and 'default'. Each queue is configured with its own 'minResources', 'maxResources', 'weight', and 'schedulingPolicy'. The 'engineering' queue uses 'fair' scheduling, while 'research' and 'default' use 'drf'. The status bar at the bottom indicates 'NORMAL' mode and the file path 'fair-scheduler.xml'.

```
1 <allocations>
2   <defaultQueueSchedulingPolicy>drf</defaultQueueSchedulingPolicy>
3
4   <queue name="engineering">
5     <minResources>2048mb,2vcores</minResources>
6     <maxResources>8192mb,8vcores</maxResources>
7     <weight>3.0</weight>
8     <schedulingPolicy>fair</schedulingPolicy>
9   </queue>
10
11  <queue name="research">
12    <minResources>1024mb,1vcores</minResources>
13    <maxResources>6144mb,6vcores</maxResources>
14    <weight>2.0</weight>
15    <schedulingPolicy>drf</schedulingPolicy>
16  </queue>
17
18  <queue name="default">
19    <minResources>512mb,1vcores</minResources>
20    <maxResources>4096mb,4vcores</maxResources>
21    <weight>1.0</weight>
```

2. Activation de le fair scheduler dans yarn-site.xml

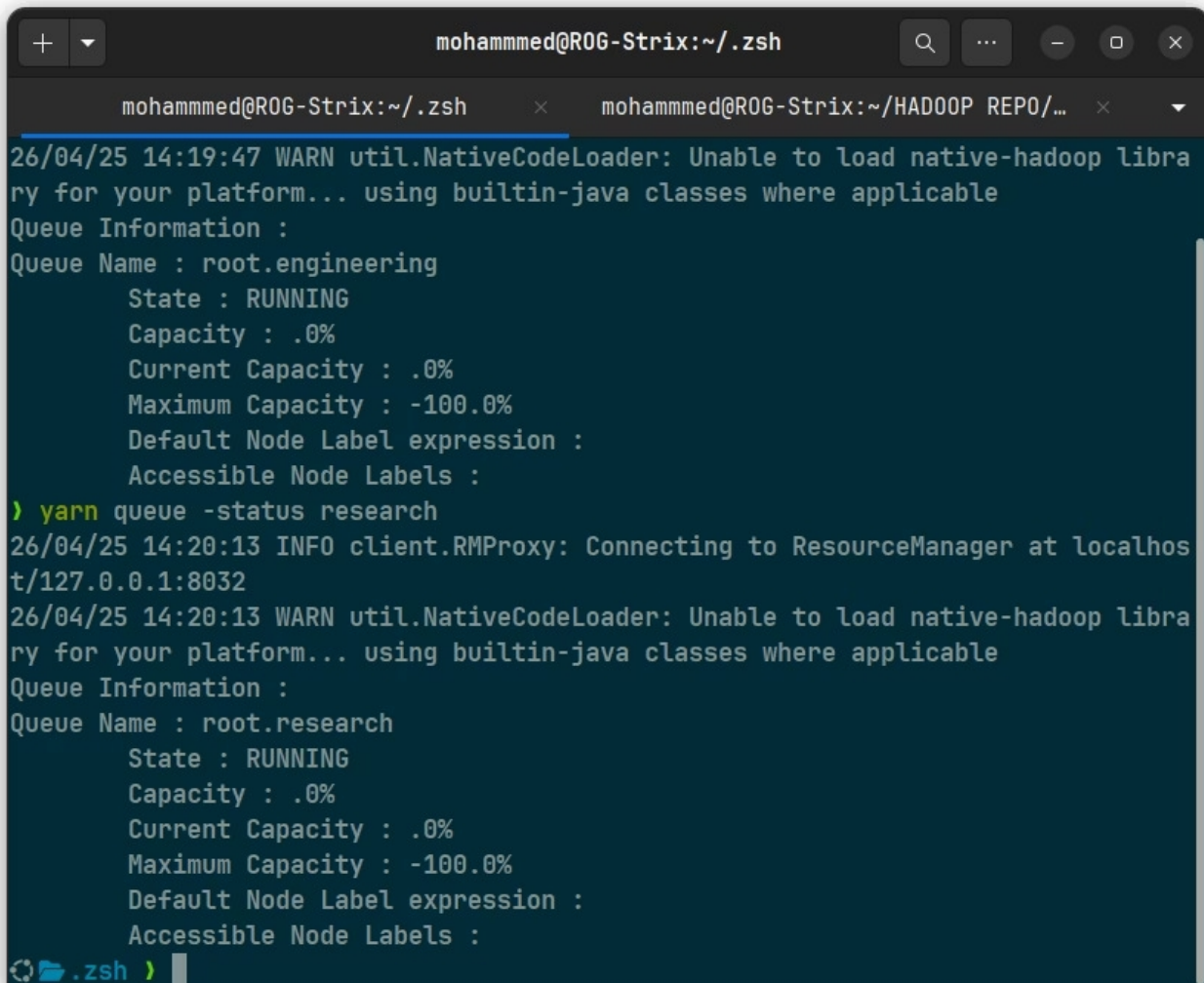
Après la création du fichier `fair-scheduler.xml`, le fichier `yarn-site.xml` est modifié afin d'activer le Fair Scheduler comme ordonnanceur principal de YARN. La fonctionnalité de préemption est également activée afin de permettre la récupération automatique des ressources lorsqu'une file d'attente manque de ressources alors que d'autres en utilisent excessivement.



```
nvim $HADOOP_CONF_DIR/yarn-site.xml
nvim $HADOOP_CONF_DIR/yarn-site.xml x mohammed@ROG-Strix:~/HADOOP REPO/... x
12
11 <property>
10 |   <name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
9 |   <value>org.apache.hadoop.mapred.ShuffleHandler</value>
8 </property>
7 <property>
6 |   <name>yarn.resourcemanager.scheduler.class</name>
5 |   <value>org.apache.hadoop.yarn.server.resourcemanager.scheduler.fair.FairS
  scheduler</value>
4 </property>
3
2 <property>
1 |   <name>yarn.scheduler.fair.allocation.file</name>
37 |   <value>/home/mohammed/hadoop/etc/hadoop/fair-scheduler.xml</value>
1
2 </property>
3
4 <property>
5 |   <name>yarn.scheduler.fair.preemption</name>
6 |   <value>true</value>
7 </property>
8 </configuration>
NORMAL <yarn-site.xml> 37/1
```

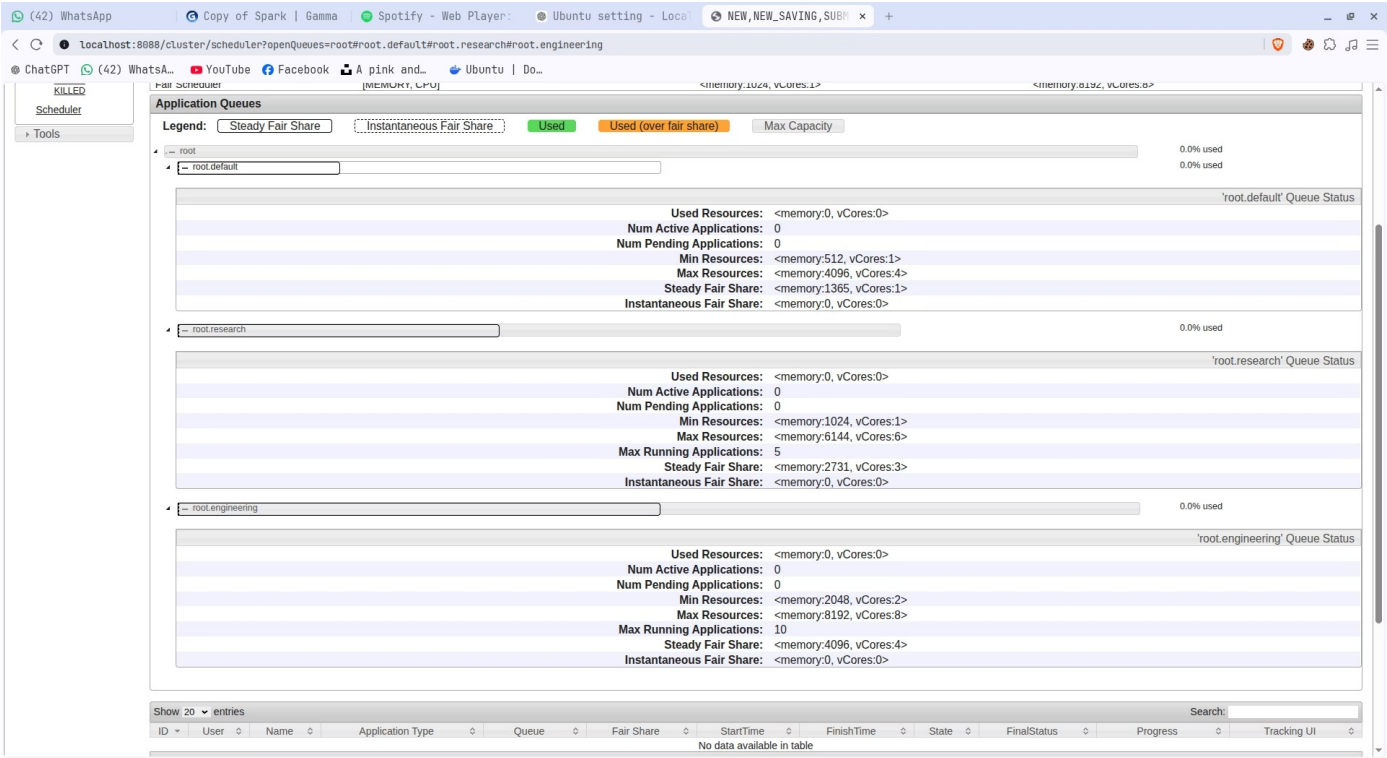
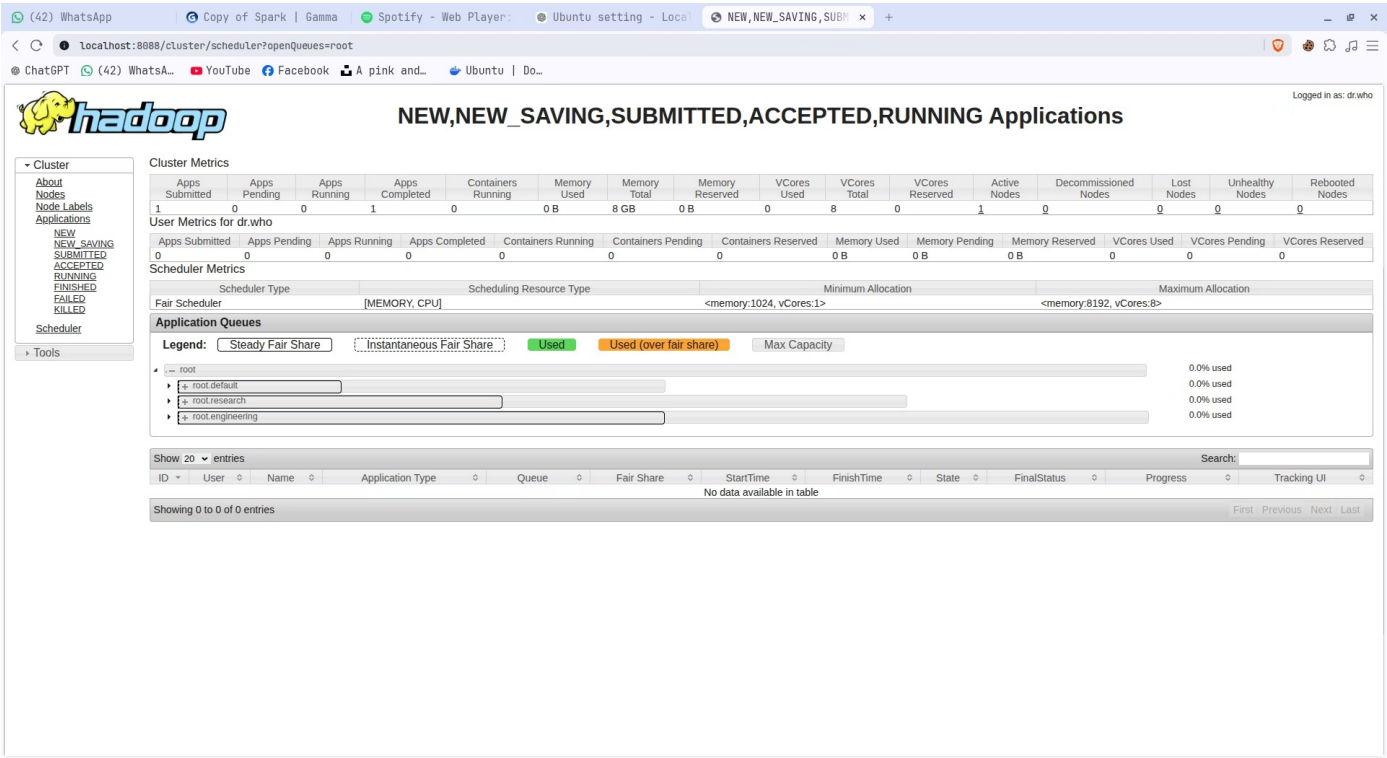
3. Redemeraer YARN et vérifier les queues

Après l'activation du Fair Scheduler, les services YARN sont redémarrés afin d'appliquer la nouvelle configuration. Les commandes `yarn queue -status engineering` et `yarn queue -status research` sont ensuite utilisées pour vérifier l'existence et l'état des files d'attente configurées. Cette vérification permet de s'assurer que les queues sont correctement reconnues par le ResourceManager et qu'elles sont prêtes à recevoir des applications.



```
mohammed@ROG-Strix:~/.zsh
mohammed@ROG-Strix:~/.zsh x mohammed@ROG-Strix:~/HADOOP REPO/... x
26/04/25 14:19:47 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Queue Information :
Queue Name : root.engineering
  State : RUNNING
  Capacity : .0%
  Current Capacity : .0%
  Maximum Capacity : -100.0%
  Default Node Label expression :
  Accessible Node Labels :
) yarn queue -status research
26/04/25 14:20:13 INFO client.RMProxy: Connecting to ResourceManager at localhost/127.0.0.1:8032
26/04/25 14:20:13 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Queue Information :
Queue Name : root.research
  State : RUNNING
  Capacity : .0%
  Current Capacity : .0%
  Maximum Capacity : -100.0%
  Default Node Label expression :
  Accessible Node Labels :
.zsh )
```

Vérification via l'interface Web de YARN



Conclusion générale

Au cours de ce travail pratique, nous avons découvert les principaux mécanismes de gestion des ressources proposés par **Apache YARN** au sein de l'écosystème Hadoop. Nous avons commencé par préparer l'environnement d'exécution en vérifiant les prérequis et en démarrant les services HDFS et YARN. Nous avons ensuite exploré les différents ordonnanceurs de ressources, notamment le **Capacity Scheduler** et le **Fair Scheduler**, en configurant des files d'attente adaptées aux besoins des utilisateurs.

La soumission et l'exécution de plusieurs jobs MapReduce nous ont permis de comprendre le fonctionnement de l'allocation des ressources et de vérifier le comportement des files d'attente configurées. Les outils de supervision, accessibles à travers les commandes YARN et l'interface Web du ResourceManager, ont également permis de suivre l'état des applications et de valider le bon fonctionnement de la plateforme.

Ce TP nous a ainsi permis d'acquérir une compréhension pratique de l'administration de YARN, de la gestion des ressources dans un cluster Hadoop et des différentes stratégies d'ordonnancement utilisées pour optimiser l'exécution des applications distribuées.